

Shield: AI-Assisted Harmful Content Intervention for Social Media Platforms

1. Problem and Product Goal

Shield is a prototype for social media Trust and Safety teams. It classifies the harassment severity of a comment, optionally using the preceding thread context, then maps the label into a deterministic product action: allow, prompt the user to reconsider, or escalate to a human moderator. The intended change from today's reactive moderation is earlier intervention. Instead of waiting for a victim to report a harmful comment, Shield can intervene before or immediately after posting.

The product goal is not automatic punishment. A moderation system that escalates everything may look safe on false negatives, but it would damage user trust by over-prompting normal criticism. In a real platform, repeated unnecessary warnings would train users to ignore safety prompts and would also create avoidable moderator workload. Shield is therefore evaluated as a product intervention, not only as a classifier.

2. System Design

The architecture separates judgement from action. The classifier produces severity, rationale, confidence, usage metadata, and abstention status. The deterministic policy table in `src/policy.py` maps severity to action: normal and low are allowed, medium prompts reconsideration, and high or critical escalates to moderation. This makes the final product decision auditable even if the classifier backend changes.

The repository now contains two backends. The keyword fallback is fully runnable without an API key and is the measured backend in this submission. The LLM backend interface is implemented with JSON-only prompting, response caching, usage tracking, and abstention handling, but no successful LLM call was measured because `SHIELD_API_KEY` was not set. For that reason, I do not claim that the LLM is better in the reported results; I only argue that the observed keyword failures show why contextual modelling is needed. This is a narrower claim than my original proposal, but it is more honest: the project currently proves the weakness of a deterministic baseline and prepares the harness needed to test an LLM next.

3. Data and Evaluation

The core evaluation set is small: 12 inspected examples, including normal criticism, direct harassment, high or critical threats, and context-sensitive cases. I kept this file separate because I did not want to silently inflate the hand-labelled set with unreviewed rows. I also added a 300-row public Jigsaw supplementary sample. Those rows are mechanically mapped from Jigsaw labels into Shield labels, so they are useful as a larger sanity check but not as human-labelled Shield ground truth. I generated an additional `candidates_to_label.csv` file with AI-drafted candidates for later human review; those rows are not merged into the core set until reviewed.

The evaluation script reports severe false-negative rate, normal-criticism false-positive rate, exact accuracy, macro F1, abstention rate, confusion matrix, lazy baseline, and context movement. I added Clopper-Pearson confidence intervals because the sample is small; the wide intervals are a useful reminder that the point estimates should not be overinterpreted. It also renames the old hand-written context labels as `legacy_handwritten_*` so they are not counted as system output.

4. Results and Critique

All current metric numbers come from `evals/results/20261004T082551Z_keyword/`. On the core set, the keyword fallback has 66.7% exact accuracy and macro F1 of 0.395. It passes the normal-criticism false-positive target on the tiny sample: 0 of 4 normal criticism cases were prompted or escalated, for 0.0% FPR. However, it fails the safety target badly: 3 of 4 high or critical cases were not escalated, for 75.0% FNR. The 95% confidence intervals are wide because the sample is tiny.

The lazy all-high baseline demonstrates why paired metrics matter. It achieves 0.0% FNR on high/critical cases, but 100.0% FPR on normal criticism. That would be a bad product because every ordinary criticism would be escalated. Shield should therefore be judged on both user protection and user friction.

The context test is now computed from code output, not hand-written labels. The keyword backend achieved 80.0% strict and lenient context movement accuracy, missing one expected context escalation. This is better than random, but still too brittle for deployment.

The supplementary 300-row public Jigsaw eval is also weak: 38.7% exact label accuracy, 5.0% normal-reference FPR, and 85.0% high/critical FNR. This confirms that the keyword fallback is not a real moderation classifier.

5. Cost and Class 5 Coverage

The cost model in `evals/results/cost_report.md` uses measured evaluation actions and ASSUMED prices. Under those assumptions, the keyword backend costs about USD 55.56 per 1,000 comments after expected human-review fallback. The LLM no-key path costs about USD 666.70 per 1,000 comments because every case abstains and escalates. These prices must be verified against provider price pages, but the exercise adds the missing platform-scale question: moderation quality must be weighed against cost per 1,000 comments and human review fallback.

6. Limitations and Future Path

The rough edges are important. The keyword fallback appears tuned to early examples: rules such as `again` with context and terms like `idiot`, `worthless`, or `hurt` line up with test cases. It also uses substring matching, so `die` can match `diet` and `kill` can match `skill`. The evaluation has one primary reviewer, a small core set, and a mechanically mapped Jigsaw supplement from long Wikipedia comments rather than short social media threads. The AI-drafted candidate set may also be optimistically biased if reviewed with a model from the same family as the future classifier.

The main difficulty I overcame was making the project auditable: replacing hand-written metric claims with scripts, cached outputs, confidence intervals, lazy baseline comparison, and a cost model. I also had to separate three different kinds of evidence that were previously mixed together: the small hand-inspected core set, the context movement tests, and the larger public Jigsaw supplement. That separation makes the results less flattering but easier to trust. The next step is to fill `candidates_to_label.csv` through human review, build a roughly 150-item core set with at least 30 normal criticism and 30 high-risk cases, then run both keyword and LLM backends with real API usage. I would also add a bias slice for identity-related language, separate sarcasm from threats, and test adversarial spellings before claiming deployment readiness. Only after that should Shield claim that contextual LLM classification improves on keywords. The product path should remain human-in-the-loop: AI triages and explains, while moderators handle severe, uncertain, and appealable cases.